

基本情報・開発体制

ジャンル	3Dアクション / スコアアタック
開発環境	C++17 / DxLib / Effekseer / Visual Studio 2022
開発体制	チーム制作 (5名)
担当役割	チームリーダー メインプログラマー 企画補助

商用ゲームエンジンを使用せず、C++とDxLibを用いてゼロから構築したフル3Dアクションゲームです。UFOを操作して地上にいる牛をビームで捕獲し、ゲージ管理と連続捕獲によるコンボ維持を駆使してハイスコアを目指します。

主要システム

■ 基本操作とリソース管理

WASD移動とマウス左クリックによるビーム照射。照射中はアプダクションゲージを消費するため、無駄撃ちを避ける状況判断が求められます。

■ コンボシステム

途切れずに連続して牛を捕獲することでスコア倍率が上昇。効率よく群れを巡る移動ルートの構築がスコア向上の要となります。

■ フィーバーモード

特定条件で出現する金の牛を捕獲すると突入。大量の牛が降り注ぎ、ゲージを消費せず一気に得点を伸ばす機会を提供します。

■ 機体強化システム

進行に応じてスキルを解放し、UFOの性能を強化。プレイヤーのプレイスタイルに応じた戦略の選択肢を持たせています。

制作における役割と主眼

チームリーダーとして全体の進捗管理やメンバー間の連携を統括しつつ、メインプログラマーとして基盤設計と当たり判定等のコアシステムを一貫して実装しました。また企画補助として、手触りの良さやゲームテンポを重視したルール調整を担当しました。

画面構成



タイトル画面：自作UIとシーン制御



プレイ画面：3D空間での吸引エフェクトと牛の配置

1. アーキテクチャ設計（共通基盤の構築）

5人体制の開発において、他メンバーの実装が円滑に進むよう、明確な責任分離と拡張性を備えた基本構造を設計しました。

■ シーン管理 (SceneManager)

全シーンを抽象基底クラス Scene として定義し、初期化・更新・描画・終了処理の一貫したライフサイクルをポリモーフィズムで実装。新シーン追加時に既存コードへ影響を与えない構造としました。

■ オブジェクト管理と遅延削除 (ObjectManager)

3Dおよび2Dの全オブジェクトを一括管理。生存フラグによる遅延削除を採用し、更新ループの実行中にコレクションが破壊されることによる不正アクセスを防止しました。

■ サービスロケータによる依存分離 (ServiceLocator)

音声、入力、リソース管理へのアクセスを一元化。グローバル変数の乱用を防ぎ、クラス間の結合度を低く保つことで変更に強い設計を確立しました。

2. 自作当たり判定システム

ゲームエンジンに頼らず、3D空間における衝突計算をゼロから数学的に実装しました。対象の形状に合わせて判定方式を最適化しています。

球体 スフィア判定 (SphereCollider)

2点間の距離の2乗と半径の和の2乗を比較。平方根の計算を省略することで処理負荷を軽減し、高速な判定を実現しました。

線分+球 カプセル判定 (CapsuleCollider)

線分と球、線分同士の最近接点を算出して接触を判定。ビーム照射の円柱領域や牛の体形に即した正確な当たり判定を行っています。

イベント駆動型の衝突検知 (ColliderManager)

単なる接触の真偽値判定にとどまらず、前フレームの接触状態との差分から3種類のイベントを自動発火する仕組みを構築しました。

イベント	発火タイミング	ゲーム内での主な処理
OnEnter	衝突した最初のフレーム	効果音再生、衝突エフェクト生成、初回ヒット処理
OnTrigger	接触継続中の全フレーム	UFO側への連続吸引処理、エフェクトの追従
OnExit	衝突判定から離脱したフレーム	吸引状態の解除、落下処理、各種状態リセット

3. 牛のAI設計とゲームバランス調整

- State/パターンによる状態管理
巨大な条件分岐の発生を防ぐため、牛の各行動（待機・移動・逃走・吸引中・消滅）を独立した状態クラスとして実装。新行動の追加時にも既存の状態ロジックを崩さない疎結合な構造を徹底しました。
- 定数ファイルによるバランス調整の集約
進行度に応じた牛の移動速度や逃走範囲の数値を GameConstants.h に一括集約。企画補助として、プログラミング知識を要せずにパラメータ調整が行える環境を整えました。

4. 堅牢なコード品質への取り組み

状態不変性の保証（const 修飾子の徹底）
内部状態を変更しない参照関数に const を徹底付与。不慮の値書き換えをコンパイル段階で検知します。

不要なメモリ複製の排除（const 参照渡し）
文字列や構造体などのサイズが大きい引数は const 参照で受け取り、不要なコピーオーバーヘッドを防止。

```
// メンバの不変性をコンパイル時に保証 VECTOR  
GetPosition() const { return position_; } float  
GetSpeed() const { return speed_; }
```

```
// 不要なオブジェクト複製を防止 void  
LoadBGM(SoundBgm bgm, const std::string&  
filename);
```

- Doxygenによるドキュメント整備とチーム開発支援
全公開メソッドに引数・戻り値・事前条件を明記したドキュメントコメントを記述。IDE上で仕様が即座に確認できるようにし、チームメンバーのコード読み込み時間を大幅に短縮しました。

5. ビルド情報・開発環境

開発言語 / 規格	C++17	開発ツール	Visual Studio 2022
使用ライブラリ	DxLib / Effekseer for DxLib	ビルド結果	0 エラー・0 警告 (Release x64)

就職活動における総括
リーダーとして開発基盤を整備し、チーム内の円滑な分業と品質管理を主導しました。また自作エンジン同等の技術力を養うため、当たり判定やAI状態遷移をゼロから実装し、警告ゼロの高精度なコードベースを維持しています。